



UNIVERSIDAD AUTÓNOMA DE OCCIDENTE

Maestría en Inteligencia Artificial y Ciencia de Datos  
Computación en la Nube

---

*Sistema de gestión de tareas en AWS*

---

**Proyecto Final**

Desarrollo de una aplicación web de alta disponibilidad

Presentado por:

**Diego Fernando Teuta Henao**

**Luis Santiago Osorio Ortiz**

Profesora:

Anabel Montero Posada

---

Mayo de 2026 • Cali, Colombia

# Índice general

|          |                                                              |           |
|----------|--------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introducción</b>                                          | <b>2</b>  |
| 1.1      | Objetivos cumplidos . . . . .                                | 2         |
| <b>2</b> | <b>Arquitectura propuesta</b>                                | <b>3</b>  |
| 2.1      | Componentes y responsabilidades . . . . .                    | 4         |
| 2.2      | Configuración de red . . . . .                               | 4         |
| 2.3      | Grupos de seguridad . . . . .                                | 5         |
| <b>3</b> | <b>Estimación de costos</b>                                  | <b>6</b>  |
| 3.1      | Evidencia y URL de la Calculadora . . . . .                  | 6         |
| <b>4</b> | <b>Resumen de la implementación</b>                          | <b>7</b>  |
| 4.1      | Fase 2 — Aplicación web monolítica (POC) . . . . .           | 7         |
| 4.2      | Fase 3 — Desacoplamiento con Amazon RDS . . . . .            | 7         |
| 4.3      | Fase 4 — Alta disponibilidad y escalado automático . . . . . | 7         |
| 4.4      | URL de acceso a la aplicación . . . . .                      | 8         |
| <b>5</b> | <b>Conclusiones</b>                                          | <b>9</b>  |
| <b>A</b> | <b>Anexos</b>                                                | <b>10</b> |
| A.1      | Capturas de evidencia . . . . .                              | 10        |
| A.2      | Anexo II — Repositorio GitHub . . . . .                      | 14        |

# Introducción

Este documento describe el diseño, despliegue y configuración de una aplicación web de gestión de tareas implementada sobre Amazon Web Services (AWS) como entregable final del curso de Computación en la Nube.

La aplicación permite a los usuarios crear, consultar, modificar y eliminar tareas. A nivel de infraestructura, la solución se compone de una capa web horizontalmente escalable distribuida en dos zonas de disponibilidad y respaldada por una base de datos relacional gestionada en una subred privada. El acceso a credenciales se realiza mediante **AWS Secrets Manager**.

## 1.1 Objetivos cumplidos

- Diseño e implementación de una VPC con cuatro subredes en dos zonas de disponibilidad.
- Despliegue de aplicación web PHP sobre Ubuntu con Apache2 y conexión a RDS MySQL.
- Migración de base de datos local a Amazon RDS en subred privada con backup habilitado.
- Almacenamiento seguro de credenciales en AWS Secrets Manager.
- Configuración de Application Load Balancer distribuyendo tráfico entre dos AZs.
- Auto Scaling Group con política de escalado por CPU Target Tracking al 50 %.
- Health check funcional en /health.php validando conectividad a la base de datos.

# Arquitectura propuesta

La arquitectura presentada en la Figura 1, está compuesta por una nube privada virtual (VPC) con segmentación en cuatro subredes —dos públicas y dos privadas— distribuidas en dos zonas de disponibilidad de la región `us-east-1`. Un **Application Load Balancer** expuesto a Internet recibe el tráfico HTTP y lo distribuye entre las instancias EC2 que componen un **Auto Scaling Group**. Las instancias ejecutan un stack LAMP con la aplicación PHP y se conectan a una instancia de **Amazon RDS MySQL** alojada en las subredes privadas. Las credenciales de la base de datos viven en AWS Secrets Manager y se leen al arranque de cada instancia mediante el rol `LabRole`.

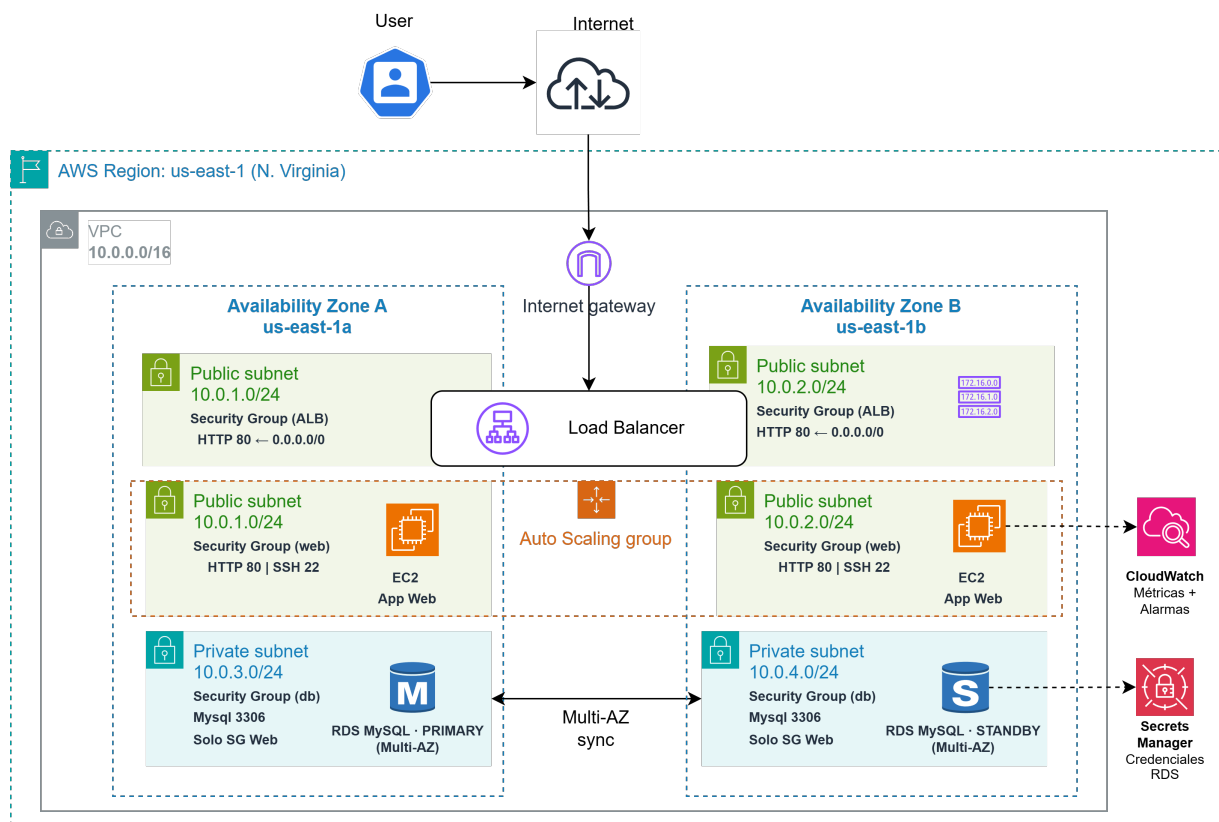


Figura 1: Arquitectura de la solución desplegada en AWS

## 2.1 Componentes y responsabilidades

La Tabla 1 describe cada recurso AWS utilizado y su función dentro de la arquitectura.

| Componente          | Servicio AWS                   | Ubicación                                | Función                                     |
|---------------------|--------------------------------|------------------------------------------|---------------------------------------------|
| Internet Gateway    | VPC - IGW                      | vpc-project-vpc                          | Punto de entrada de tráfico desde Internet. |
| Load Balancer       | Application Load Balancer      | Subredes públicas (2 AZ)                 | Distribuye tráfico entre instancias EC2.    |
| Servidores web      | Amazon EC2 (t3.micro)          | public1-us-east-1a<br>public2-us-east-1b | Alojan la aplicación web.                   |
| Escalado automático | EC2 Auto Scaling Group         | Ambas AZs                                | Escala instancias según demanda de CPU.     |
| Base de datos       | Amazon RDS MySQL (db.t3.micro) | Subredes privadas                        | Almacena datos de la aplicación.            |
| Gestión de secretos | AWS Secrets Manager            | us-east-1                                | Almacena credenciales de la base de datos.  |
| Red virtual         | Amazon VPC (10.0.0.0/16)       | us-east-1                                | Aislamiento de red de todos los recursos.   |

Cuadro 1: Componentes de la arquitectura AWS.

## 2.2 Configuración de red

La Tabla 2 muestra la segmentación de red implementada en la VPC. La arquitectura utiliza subredes públicas y privadas para la separación entre la capa de aplicación y la capa de datos.

| Subred           | CIDR        | Zona de disponibilidad | Tipo                  |
|------------------|-------------|------------------------|-----------------------|
| subnet-publica-1 | 10.0.1.0/24 | us-east-1a             | Pública — EC2 y ALB   |
| subnet-publica-2 | 10.0.2.0/24 | us-east-1b             | Pública — EC2 y ALB   |
| subnet-privada-1 | 10.0.3.0/24 | us-east-1a             | Privada — RDS         |
| subnet-privada-2 | 10.0.4.0/24 | us-east-1b             | Privada — réplica RDS |

| Subred | CIDR | Zona de disponibilidad | Tipo |
|--------|------|------------------------|------|
|--------|------|------------------------|------|

Cuadro 2: Configuración de subredes de la VPC.

## 2.3 Grupos de seguridad

La Tabla 3 describe los *Security Groups* configurados para controlar el tráfico entre los componentes de la arquitectura, aplicando el principio de mínimo privilegio.

| Security Group | Regla de entrada               | Puerto | Origen/Destino        |
|----------------|--------------------------------|--------|-----------------------|
| <b>SG-alb</b>  | HTTP desde Internet            | 80     | 0.0.0.0/0             |
| <b>SG-ec2</b>  | HTTP desde el ALB              | 80     | SG-alb                |
| <b>SG-ec2</b>  | SSH desde IP del administrador | 22     | IP administrador / 32 |
| <b>SG-rds</b>  | MySQL desde servidores web     | 3306   | SG-ec2                |

Cuadro 3: Reglas de seguridad de la arquitectura.

# Estimación de costos

La estimación se elaboró en la **AWS Pricing Calculator** para la región `us-east-1` con un horizonte de 12 meses. La estimación detallada por servicio se presenta en la Tabla 4.

| Servicio               | Configuración                                                           | Qty | USD/mes  | USD/12 m   |
|------------------------|-------------------------------------------------------------------------|-----|----------|------------|
| Amazon RDS for MySQL   | db.t3.micro, 20 GB gp2, Multi-AZ, On-Demand                             | 1   | \$69,57  | \$834,84   |
| Amazon EC2             | 2 x t3.micro Ubuntu Pro, Compute Savings Plans 3yr No Upfront, EBS 8 GB | 2   | \$12,08  | \$144,96   |
| Elastic Load Balancing | 1 Application Load Balancer                                             | 1   | \$18,47  | \$221,64   |
| AWS Secrets Manager    | 1 secreto · 100 000 API calls/mes                                       | 1   | \$0,90   | \$10,80    |
| AWS Data Transfer      | 10 GB/mes tráfico saliente a Internet                                   | 1   | \$0,90   | \$10,80    |
| Amazon CloudWatch      | Métricas básicas, 5 alarmas y 1 dashboard                               | 1   | \$2,00   | \$24,00    |
| TOTAL                  |                                                                         |     | \$103,92 | \$1 247,04 |

Cuadro 4: Estimación de costos AWS para la arquitectura propuesta en `us-east-1`.

## 3.1 Evidencia y URL de la Calculadora

URL pública de la estimación en AWS Pricing Calculator (completar tras exportar):

<https://calculator.aws/#/estimate?id=a57f9c9ef12f54edaab7269ad81c5bfe0c57a1c6>

# Resumen de la implementación

El despliegue se realizó en tres fases progresivas siguiendo la estructura propuesta.

## 4.1 Fase 2 — Aplicación web monolítica (POC)

Se creó la VPC con cuatro subredes (dos públicas y dos privadas) y un Internet Gateway. Se lanzó una instancia EC2 `t3.micro` Ubuntu 26.04 LTS en la subred pública `us-east-1a` con un script `user-data` que instala Apache, PHP, MariaDB local y despliega la aplicación. La instancia quedó accesible vía HTTP en su IP pública, sirviendo el CRUD de tareas contra la base de datos local. Este montaje sirvió como prueba de concepto antes de desacoplar la capa de datos.

## 4.2 Fase 3 — Desacoplamiento con Amazon RDS

Se creó un DB Subnet Group con las dos subredes privadas y se provisionó una instancia Amazon RDS MySQL `db.t3.micro` Multi-AZ. Sus credenciales se almacenaron en AWS Secrets Manager bajo el identificador `taskcloud/db`. La base de datos local se migró al RDS mediante `mysqldump`. La aplicación se reconfiguró para leer las credenciales desde Secrets Manager al arranque, eliminando cualquier secreto del código fuente.

## 4.3 Fase 4 — Alta disponibilidad y escalado automático

A partir de la EC2 configurada en la Fase 3 se generó una AMI (`taskcloud-ami-v1`) que sirvió como base para un Launch Template. Se creó un Application Load Balancer expuesto a Internet con listeners HTTP 80, asociado a un Target Group con health check sobre `/health.php`. Un Auto Scaling Group con `min=1`, `desired=2`, `max=4` distribuye instancias entre las dos AZ. La política de escalado es *Target Tracking* sobre CPU promedio al 50 %. Se ejecutaron pruebas de carga con `loadtest` para validar que el ASG escala hacia arriba bajo presión y vuelve al *baseline* cuando la carga cesa.



## 4.4 URL de acceso a la aplicación

La aplicación está disponible a través del DNS público del Application Load Balancer:

```
taskcloud-alb-2044196625.us-east-1.elb.amazonaws.com
```

# Conclusiones

La construcción del sistema de gestión de tareas demostró que es posible desplegar una aplicación web usando solamente los servicios de AWS. La separación de la capa de datos hacia Amazon RDS y la posterior introducción del Application Load Balancer + Auto Scaling Group materializan los principios de alta disponibilidad y escalabilidad horizontal de forma incremental.

El costo total estimado para 12 meses se mantiene en niveles compatibles con el crédito disponible en AWS Academy, evidenciando que las decisiones de optimización (Multi-AZ en RDS, `t3.micro` en EC2) son apropiadas.

Finalmente, el uso de AWS Secrets Manager permitió eliminar las credenciales del código fuente y la microsegmentación por Security Groups (SG referenciando SG) ofrece un nivel de seguridad robusto.

# Anexos

## A.1 Capturas de evidencia

Las siguientes capturas se incluyen como anexos visuales del despliegue:

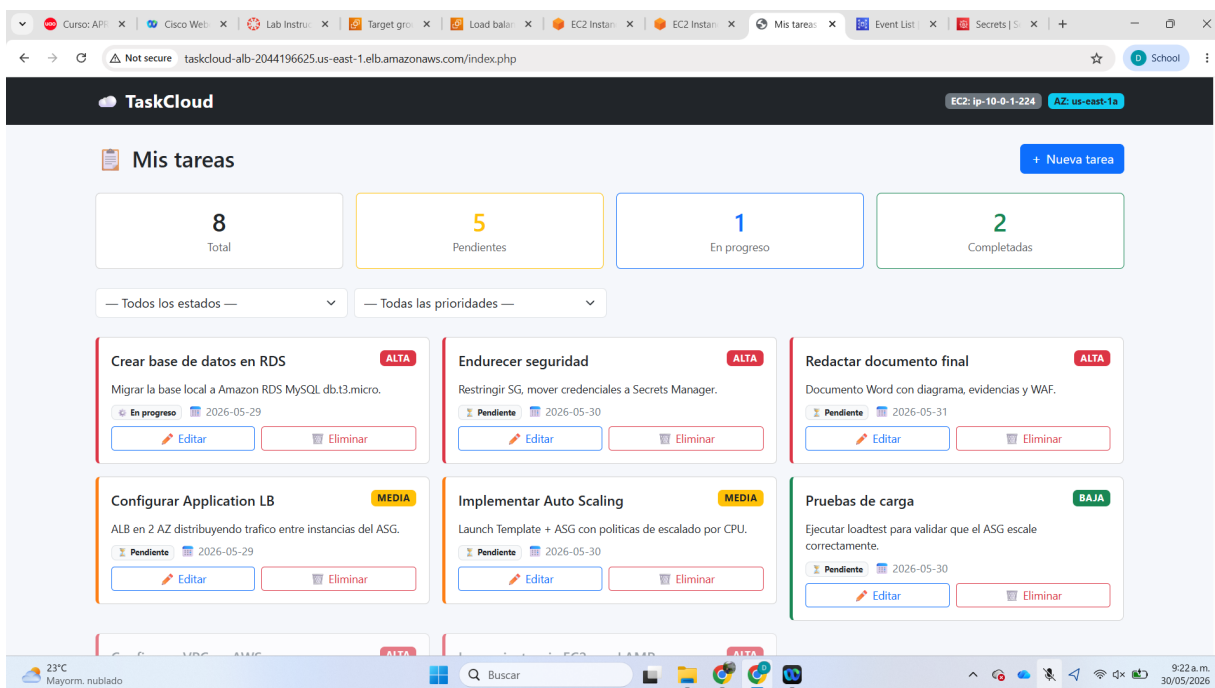


Figura 2: Aplicación funcionando desde el ALB.

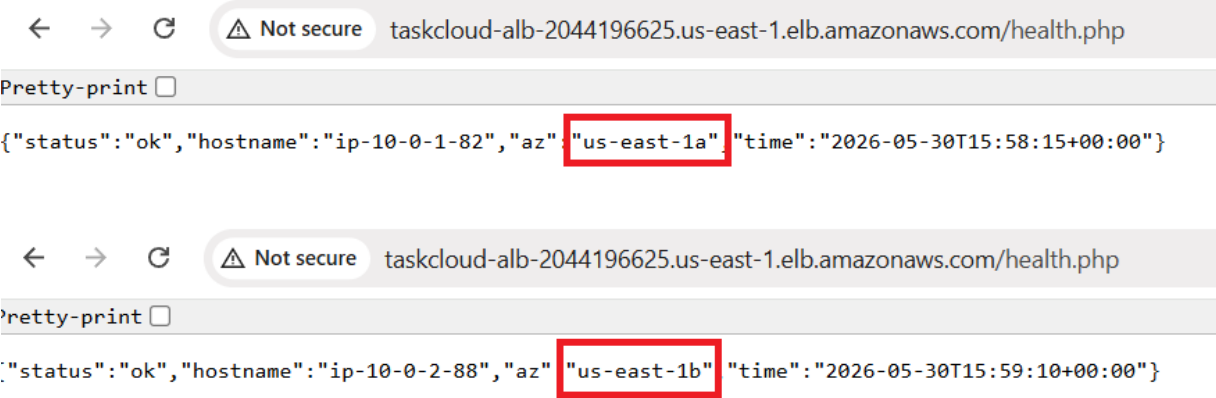


Figura 3: Balanceo entre AZs

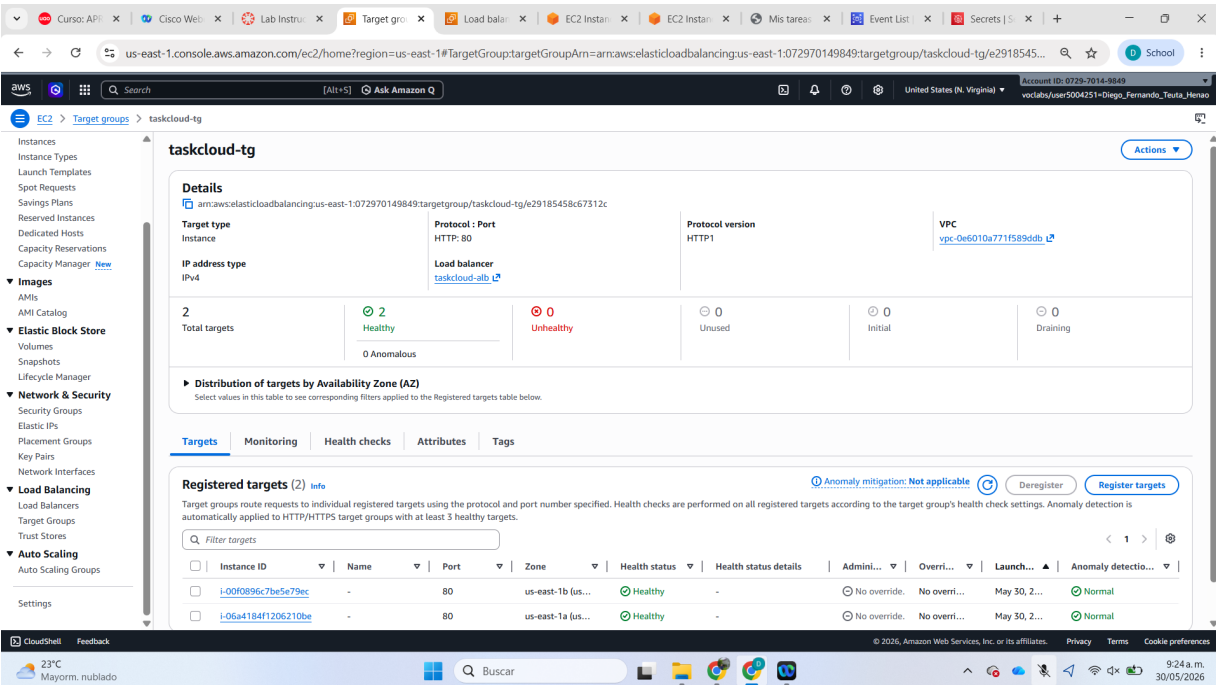


Figura 4: Target Group con instancias Healthy

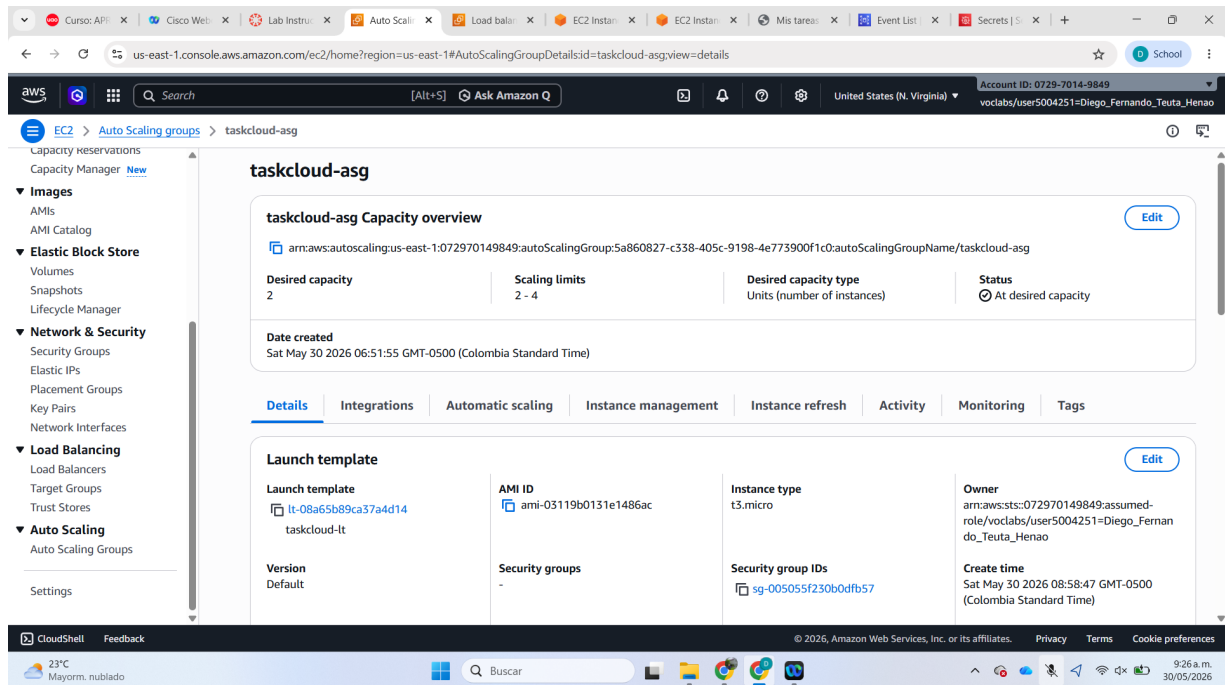


Figura 5: Instancias EC2 administradas por el Auto Scaling Group.

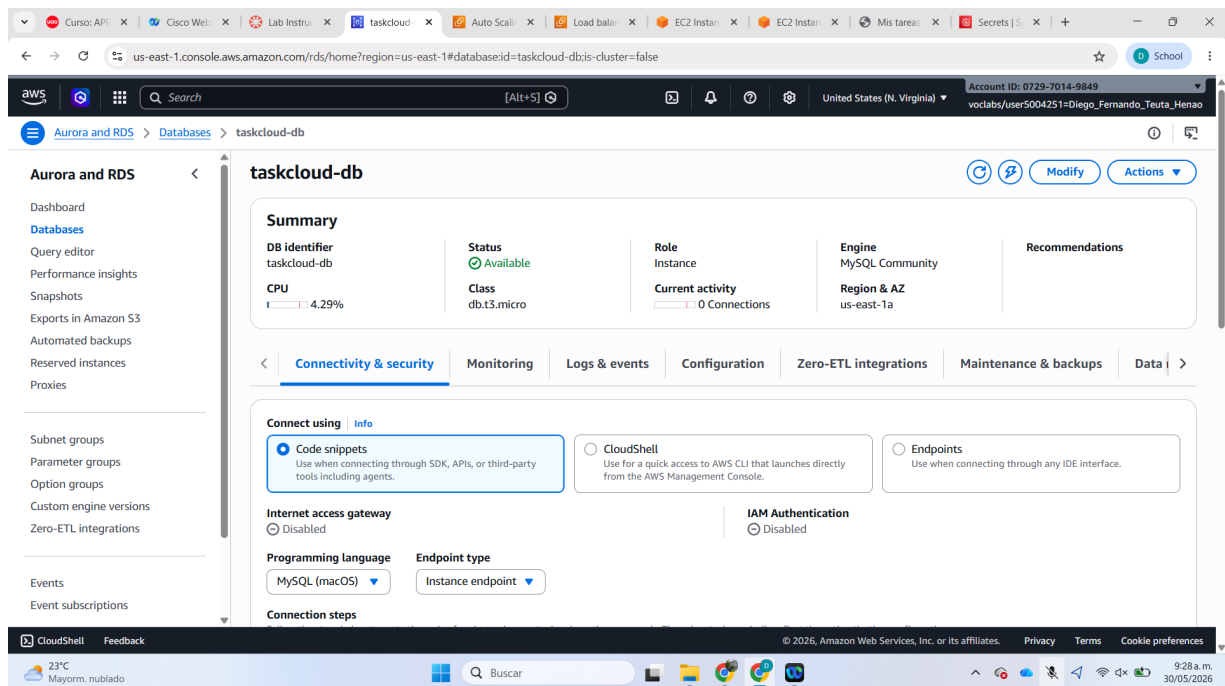


Figura 6: RDS disponible mostrando el endpoint de conexión.

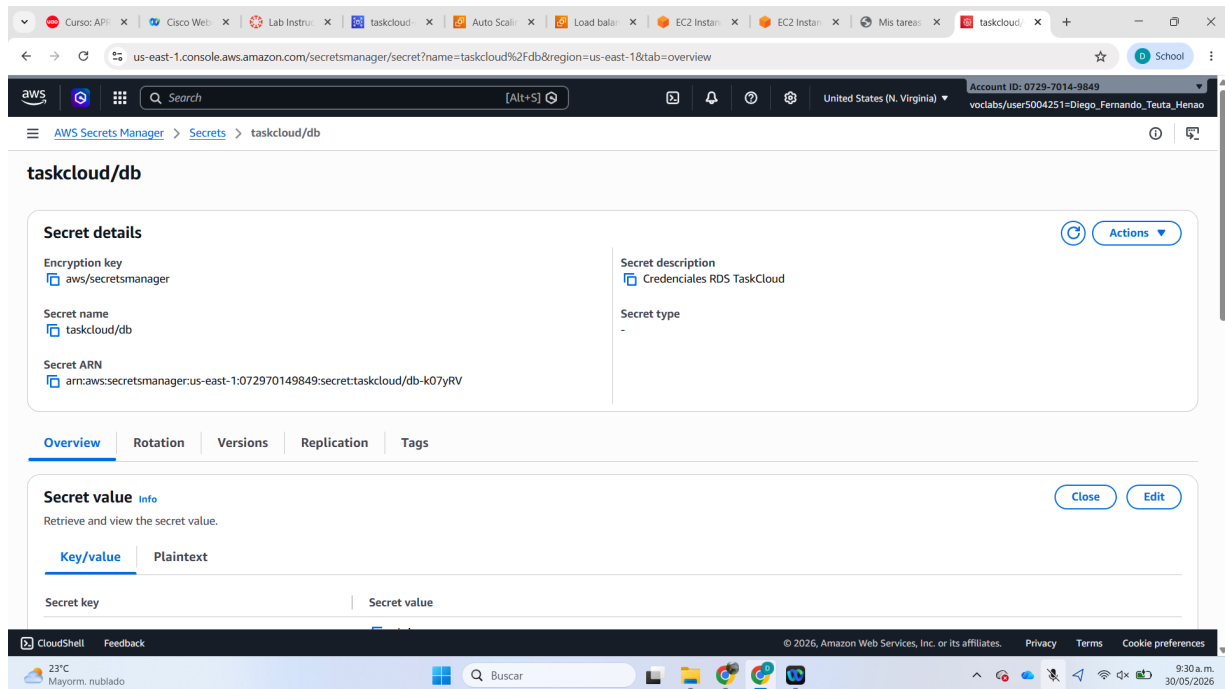


Figura 7: Secrets Manager creado con los campos host, username, dbname.

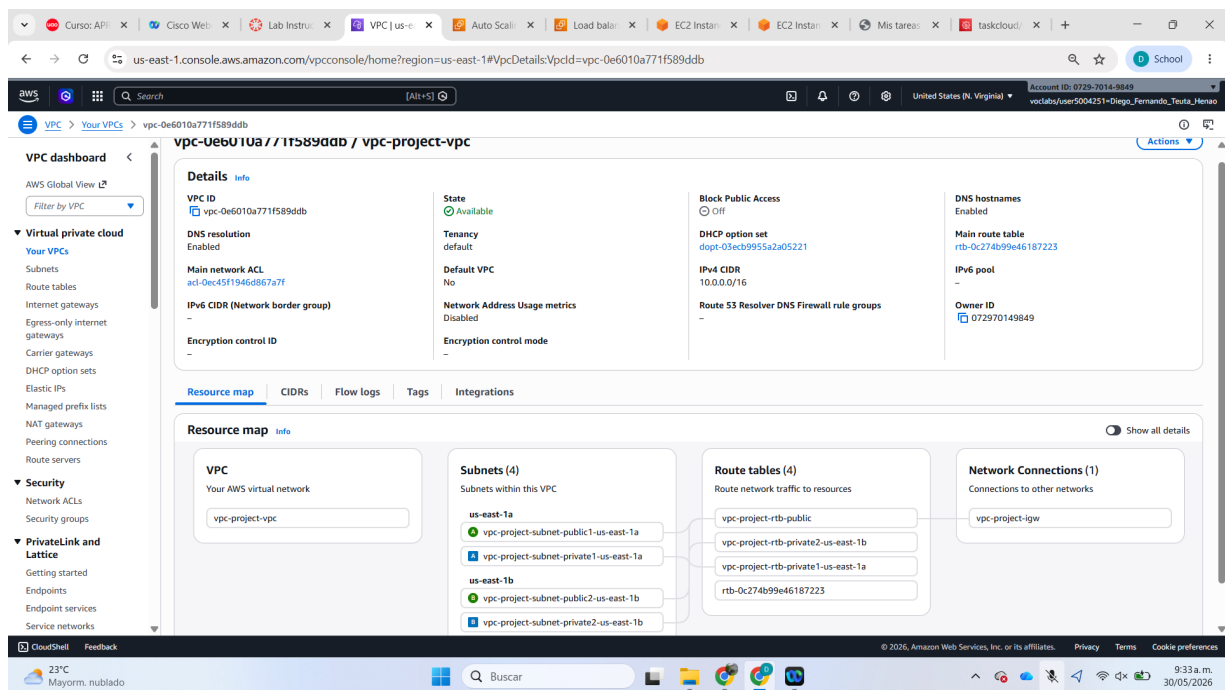


Figura 8: Resource map mostrando las subredes y tablas de rutas.



(a) CPU - prueba de estrés

(b) Escalado automático - umbral del 5 %.

Figura 9: Prueba de carga y respuesta del Auto Scaling Group.

## A.2 Anexo II — Repositorio GitHub

El código fuente completo está disponible en:

<https://github.com/diegoteuta/taskcloud-aws>